



Programação Avançada 2025-26

10 Grafos | Algoritmo Dijkstra

Bruno Silva, Patrícia Macedo

Sumário

- Algoritmo Dijkstra;
- Implementação;
- Exercícios.

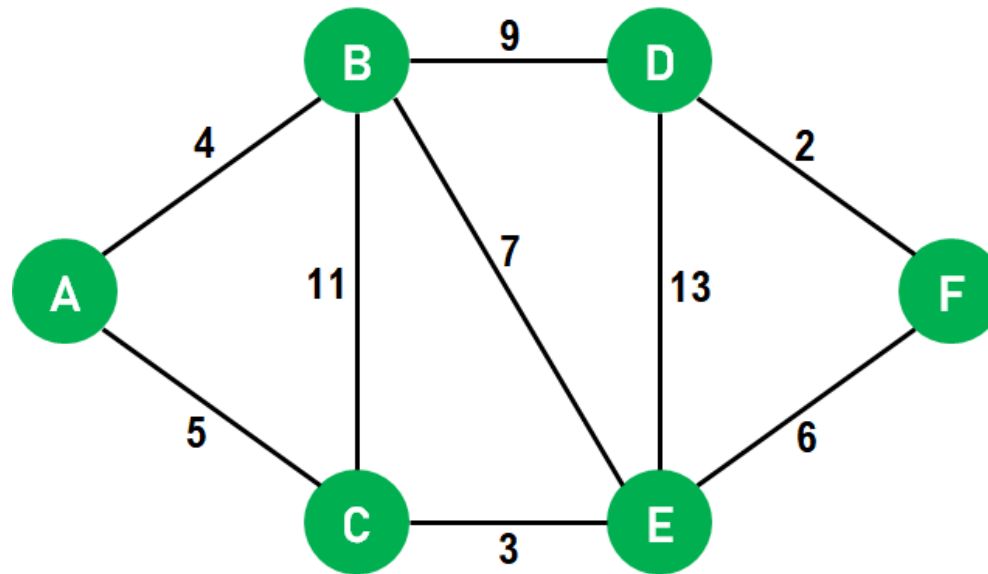
Grafos | Algoritmo Dijkstra

- O algoritmo Dijkstra é utilizado na teoria de grafos para encontrar o **caminho de menor custo entre dois vértices num grafo valorado**.
- Foi publicado por [Edsger W. Dijkstra](#) em 1956 e é amplamente utilizado em aplicações de otimização de rotas, como em sistemas de navegação, redes de computadores e logística.
- No entanto, **pode não funcionar corretamente em grafos com arestas de peso negativo**, pois não foi projetado para este efeito (acumula a soma dos pesos das arestas). Para grafos com arestas de peso negativo, outros algoritmos, como o algoritmo de *Bellman-Ford*, são mais apropriados.

💡 A pronúncia aproximada em português para Edsger Dijkstra é "*étsrrar déikstra*".

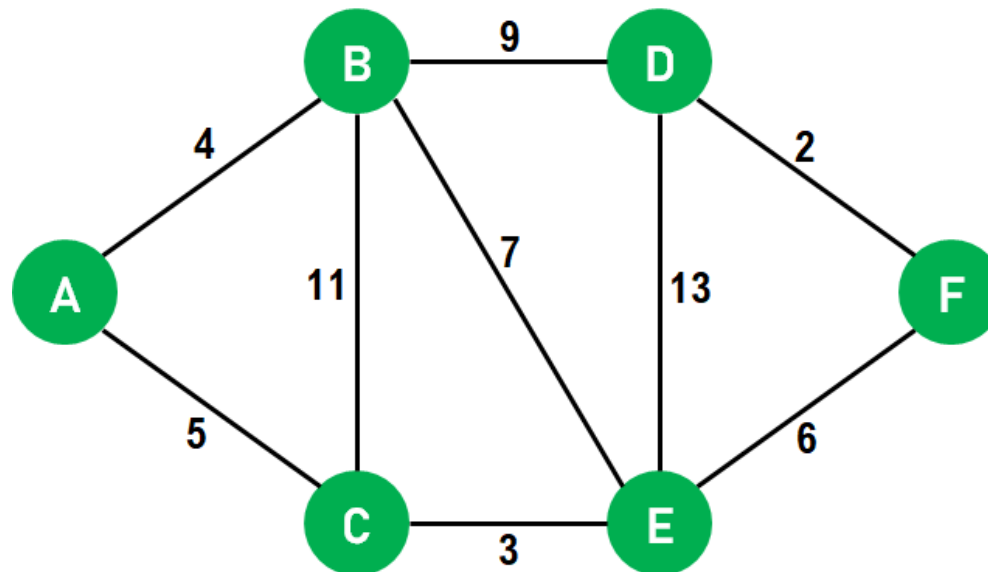
Grafos | Algoritmo Dijkstra

Qual o caminho mais curto entre **C** e **D**?



Grafos | Algoritmo Dijkstra

Qual o caminho de menor custo entre C e D?



{C, E, F, D} com um custo de $3 + 6 + 2 = 11$.

Grafos | Algoritmo Dijkstra

Na prática, o algoritmo Dijkstra **calculará o caminho de menor custo entre um vértice de origem e todos os vértices do grafo!** A informação produzida pelo algoritmo para **vértice de origem = C** será a seguinte:

Vértice	Custo	Predecessor
A	5	C
B	9	A
C	0	
D	11	F
E	3	C
F	9	E

Grafos | Algoritmo Dijkstra

A partir desta informação, como posso saber o *menor custo* e o *caminho* entre o vértice de origem **C** e vértice de destino **D** ?

Vértice	Custo	Predecessor
A	5	C
B	9	A
C	0	
D	11	F
E	3	C
F	9	E

Grafos | Algoritmo Dijkstra



Custo



Caminho

(predecessores) D -> F -> E -> C (⚠ "inverter" caminho)

Vértice	Custo	Predecessor
A	5	C
B	9	A
C	0	
D	11	F
E	3	C
F	9	E

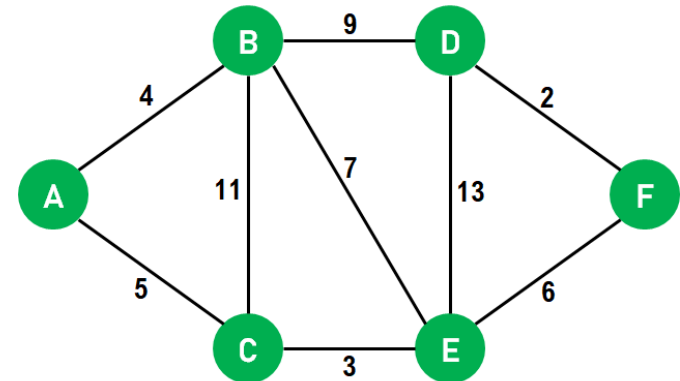


Obteremos {C, E, F, D} com um custo de 11.

Grafos | Algoritmo Dijkstra

Na realidade, esta tabela dá-nos a informação do **caminho de menor custo** entre **C** e qualquer outro vértice do grafo. Verifique !

Vértice	Custo	Predecessor
A	5	C
B	9	A
C	0	
D	11	F
E	3	C
F	9	E



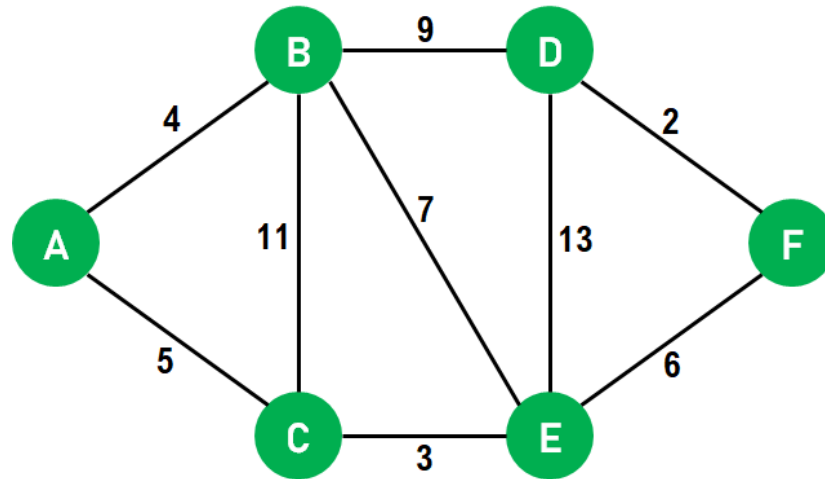
Grafos | Algoritmo Dijkstra

⚠ A tabela anterior só fornece informação para qualquer caminho que tenha o vértice **C** como *vértice de origem*.

- Se quiser saber qual o caminho de menor custo entre, e.g., **A** e **F**, esta tabela não é válida!
 - Terá de ser gerada nova tabela (pelo algoritmo Dijkstra), com vértice de origem = **A** ! *Capisce ?*

Grafos | Algoritmo Dijkstra

Qual o caminho mais curto entre **C** e **D**?



{C, B, D} ou {C, E, D}

➡ O algoritmo também nos poderá fornecer este tipo de informação se for atribuído um peso de **1** a cada aresta.

Grafos | Algoritmo Dijkstra

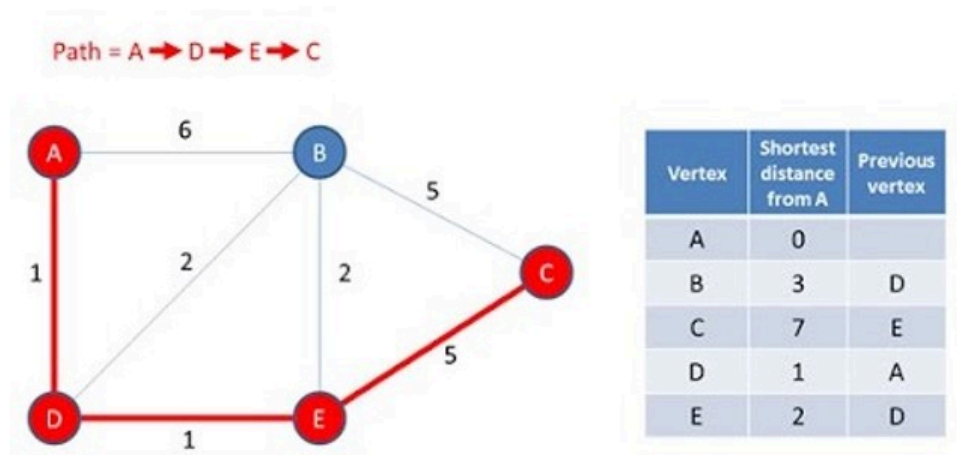
Algoritmo em linguagem natural:

```
function Dijkstra(Graph, Source):  
    create an empty set of unvisited vertices  
    for each vertex V in the Graph:  
        assign infinite cost to V  
        assign null predecessor to V  
        add V to the set of unvisited vertices  
    assign cost 0 to the Source vertex  
    while there are unvisited vertices:  
        select the unvisited vertex V with the smallest current cost  
        mark V as visited (remove from the set)  
        for each neighbor W of V:  
            if the current cost of [V + weight of edge (V, W)] is less  
               than the current cost of W:  
                update the cost of W  
                set V as the predecessor of W  
    return the result (costs and predecessors)
```

Grafos | Algoritmo Dijkstra

Exemplo de funcionamento:

<https://www.youtube.com/watch?v=pVfj6mxhdMw>

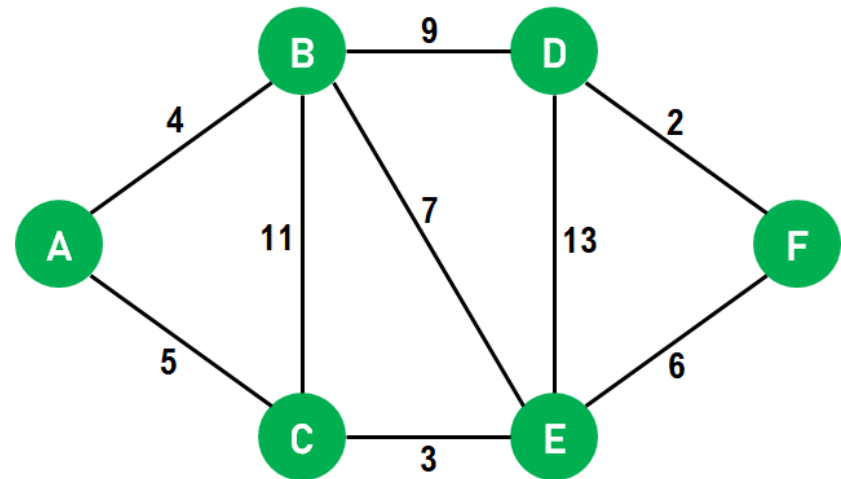


Grafos | Algoritmo Dijkstra

Exercício #1

Aplique o algoritmo *Dijkstra*, com `Origem = C`.

Vértice	Custo	Predecessor
A		
B		
C		
D		
E		
F		

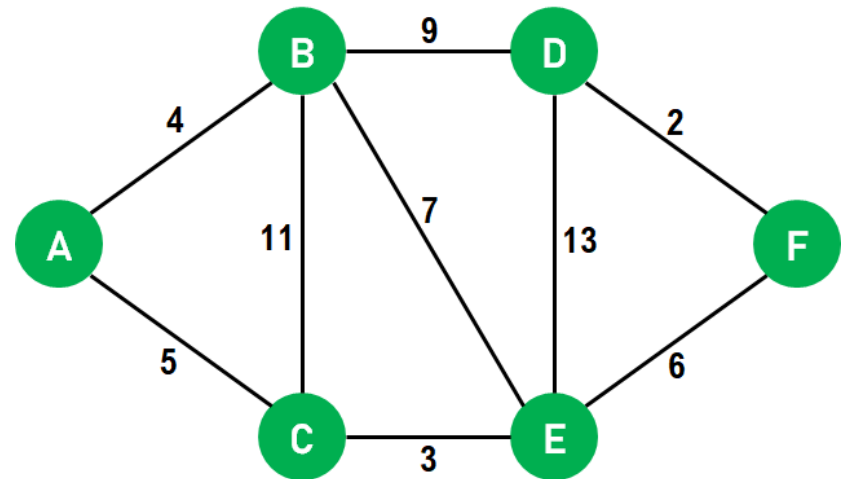


Grafos | Algoritmo Dijkstra

Exercício #2

Aplique o algoritmo *Dijkstra*, com `Origem = F`.

Vértice	Custo	Predecessor
A		
B		
C		
D		
E		
F		

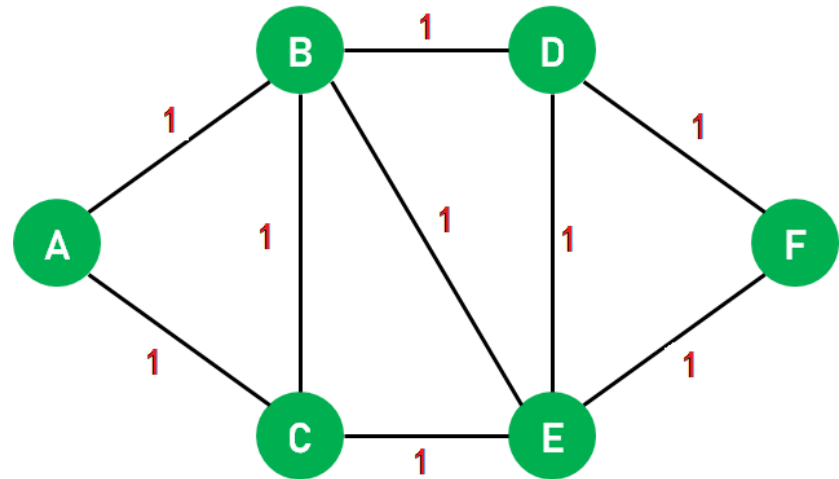


Grafos | Algoritmo Dijkstra

Exercício #3 (AUTÓNOMO)

Aplique o algoritmo *Dijkstra*, com `Origem = E`, mas com peso `1` em todas as arestas ➡ obtenção de caminhos mais curtos.

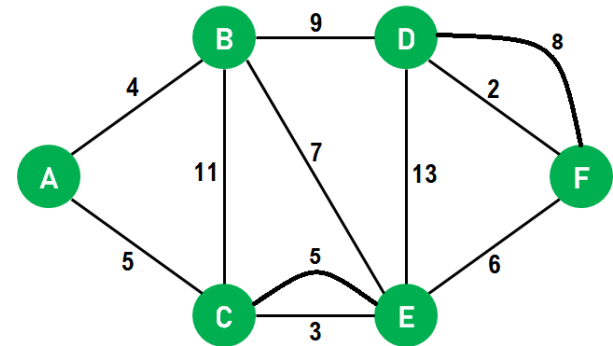
Vértice	Custo	Predecessor
A		
B		
C		
D		
E		
F		



Grafos | Algoritmo Dijkstra

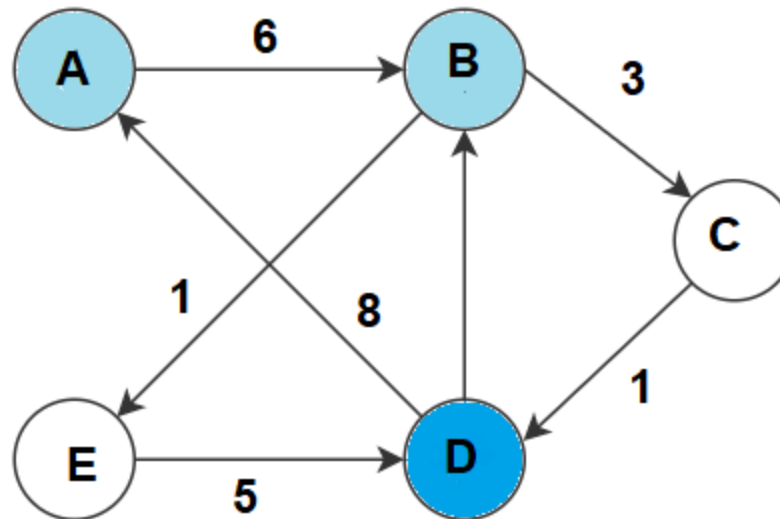
⚠️ Caso o grafo possua *arestas paralelas*, se necessitarmos saber o caminho de menor custo (incluindo as arestas), teremos de estender a tabela para conter esta informação, e.g.:

Vértice	Custo	Predecessor	via Aresta
A			
B			
C			
D			
E			
F			



Grafos | Algoritmo Dijkstra

⚠ No caso de grafos orientados (dígrafos), os vértices "vizinhos" (descritos no algoritmo) de **V** serão apenas os obtidos através das suas *arestas de saída*.



Grafos | Algoritmo Dijkstra

IMPLEMENTAÇÃO sobre o ADT Graph será realizada na aula Laboratorial - Lab 6

Algoritmo Dijkstra | Conclusão

- O algoritmo *Dijkstra* é um algoritmo ganancioso ("greedy"), com uma complexidade algorítmica de $O(V^2)$, onde V é o número de vértices.
 - Portanto, o "trabalho" aumentará quadraticamente relativamente ao número de vértices no grafo.
- O algoritmo Dijkstra só trabalha corretamente com arestas de peso positivo.
 - Nestes casos teremos de utilizar outros algoritmos.

Algoritmo Dijkstra | Referências

- https://en.wikipedia.org/wiki/Edsger_W._Dijkstra
 - https://en.wikipedia.org/wiki/Dijkstra's_algorithm
-
- Vídeo YouTube - Explaining Dijkstra
 - <https://www.geeksforgeeks.org/dijkstras-shortest-path-algorithm-greedy-algo-7/>